

GVars Integration into Groovy: Final Analysis

Targeting Groovy 6, JDK 17 minimum. Assumes the Groovy async proposal (PR #2437 / GROOVY-9381) is merged. Assumes STM, CSP, and Remoting are dropped regardless of option chosen (unmaintained external dependencies, low adoption).

Background

What we're bringing across

After pruning STM (Multiverse, unmaintained), CSP (JCSP, niche), and Remoting (Netty, better done with modern alternatives), the GVars functionality that remains is:

- **Parallel collections** — `eachParallel`, `collectParallel`, `findAllParallel`, `withPool()`, etc.
- **Dataflow** — single-assignment variables, channels (queues, broadcasts), operator networks, pipelines, `select/guard`
- **Actors** — `DefaultActor`, `DynamicDispatchActor`, `StaticDispatchActor`, `ReactiveActor`, `@ActiveObject` AST transform
- **Agents** — thread-safe mutable state containers
- **Core infrastructure** — Pool interface, `ForkJoinPool/ExecutorService` management, async messaging engine, configuration

JDK parallel streams overlap

JDK parallel streams (`stream().parallel()`) now cover the same core operations as GVars parallel collections (`each`, `collect`, `filter`, `find`, `reduce`, `groupBy`). The GVars value-add in this area is primarily **Groovy ergonomics** (category methods, `withPool()` scoping, `TransparentParallel`) and **pool isolation** (dedicated pools rather than the shared common `ForkJoinPool`). Internally, a `groovy-parallel-collections` module could delegate to JDK parallel streams rather than porting the full GVars parallel array infrastructure.

The other modules (dataflow, actors, agents) have **no JDK equivalent** and must bring the GVars code regardless of option.

@Parallel is separable

The existing `@Parallel` annotation and `ParallelASTTransformation` live in Groovy core and are unaffected by the option choice. Enhancing the transform to use a pool-backed implementation when available is a codegen change in core, not a module structure decision. It can proceed independently on its own timeline.

Groovy async integration

The Groovy async proposal introduces `Awaitable`, `AsyncChannel`, `AsyncScope`, and the `AwaitableAdapter` SPI. How GVars-derived modules integrate with this depends on the option chosen.

Option 1: GVars as a Single Groovy Subproject

Bring the GVars repository in as `groovy-gvars`. Update to Groovy 6 / JDK 17. Strip the dead weight. Ship as one module.

What users get

- The full GVars API in one dependency
- Existing GVars documentation, examples, and mental model carry over
- PGroup as the central integration point stays intact

Pros

- **Lowest migration effort** — minimal refactoring of GVars internals
- **Actor↔dataflow coupling stays intact** — no need to untangle the shared `MessageStream` base or `PGroup` facade
- **Familiar** — existing GVars users know the API
- **No external dependencies** after pruning STM/CSP/Remoting

Cons

- **Monolith** — users wanting parallel collections pull in actors, dataflow, agents; no way to take only what you need
- **Requires Groovy runtime** — Java users must have Groovy on the classpath; Closure-based API is not idiomatic Java
- **Async overlap is messy** — GVars' `Promise` competes with `Awaitable`; `DataflowQueue` competes with `AsyncChannel`; `PGroup` competes with `AsyncScope`. Adapters are bolted on after the fact.
- **Parallel collections largely duplicates JDK streams** for Java users, but they can't use the JDK alternative without also pulling in the Groovy sugar they don't need
- **Versioning tied to Groovy releases** — every GVars change ships with a Groovy release

Async integration approach

`AwaitableAdapter` implementations would be added inside the single module to bridge GVars' `Promise` / `DataflowVariable` /actor replies to `Awaitable`. This works but creates a bidirectional dependency: GVars depends on Groovy core (for `Closure` etc.), and the adapters reference `Awaitable` (also in core). The two systems coexist but don't share a unified design.

Option 2: Decomposed into Groovy Modules

Break GVars into focused Groovy subprojects: `groovy-parallel-core`, `groovy-parallel-collections`, `groovy-dataflow`, `groovy-actors`, `groovy-agents`. All depend on Groovy core.

Proposed modules (priority order)

Module	Contents	Size	Effort
<code>groovy-parallel-core</code>	Pool, Scheduler, <code>AsyncMessagingCore</code> , <code>MessageStream</code> , <code>GParsConfig</code>	~25 files	Low
<code>groovy-parallel-collections</code>	<code>GParsPool</code> , <code>withPool()</code> , <code>eachParallel</code> , <code>collectParallel</code> , <code>fork/join</code>	~30 files (less if delegating to JDK streams)	Low
<code>groovy-dataflow</code>	<code>DataflowVariable</code> , channels, operators, pipelines, Kanban	~75 files	Medium
<code>groovy-actors</code>	Actor types, <code>@ActiveObject</code> , slimmed <code>PGroup</code>	~35 files	Medium
<code>groovy-agents</code>	Agent (could fold into core given ~4 files)	~4 files	Trivial

What users get

- Pick only what you need — parallel collections without actors, dataflow without agents
- Full Groovy-idiomatic API: `withPool { items.eachParallel { } }`
- Deep integration with Groovy async via `AwaitableAdapter`
- AST transforms: `@ActiveObject`, `@AsyncFun`, enhanced `@Parallel`

Pros

- **Modular** — fine-grained dependency management
- **Deep Groovy async integration** — each module's types can directly implement or adapt to `Awaitable`; `DataflowVariable` is natively `await`-able
- **Modules evolve independently** — a dataflow fix doesn't require re-releasing actors
- **Can delegate parallel collections to JDK streams internally** — reduces codebase, inherits JDK optimizations
- **No external dependencies**
- **Familiar Groovy API** for existing GVars users

Cons

- **All modules depend on Groovy core** — Java users need the Groovy runtime on the classpath even if they only call Java-signature methods
- **Closure-based API is not idiomatic Java** — Java users deal with `groovy.lang.Closure` rather than `java.util.function` interfaces
- **Not usable from Kotlin/Scala** without Groovy runtime and awkward Closure interop
- **Medium refactoring effort** — must decouple `PGroup`, move `MessageStream` to core, remove serial/remote infrastructure

Async integration approach

Each module implements `AwaitableAdapter` directly. `DataflowVariable` could implement `Awaitable` as a first-class interface. Actor `sendAndPromise()` returns an `Awaitable`. This is the deepest, most native integration — Groovy async and GVars-derived modules share the same type system.

Option 3: Java-First Modules with Optional Groovy

Same modular decomposition as Option 2, but with a dual API: the Java API uses `java.util.function` interfaces and `CompletionStage` for async results. Groovy sugar (categories, `withPool()`, Closure overloads, AST transforms, `AwaitableAdapters`) lives in the same JAR but Groovy is declared as `compileOnly` — an optional dependency that Java users never pull transitively. JVM lazy class loading ensures the Groovy classes are never loaded unless referenced.

How it works

- Java API classes use `Consumer<T>`, `Function<T,R>`, `Predicate<T>` instead of `Closure`
- Async results are `CompletableFuture<T>` / `CompletionStage<T>` — the JDK standard
- Groovy sugar classes live in a `.groovy` sub-package within the same module
- Groovy is `compileOnly` — not transitive to consumers
- `AwaitableAdapter` implementations are in the JAR but only loaded by Groovy's async runtime (which is only present if Groovy is on the classpath)
- A CI fitness test compiles and runs Java-only tests with Groovy excluded from the classpath, catching accidental coupling

Module structure (same 5 modules as Option 2)

```
groovy-parallel-core/
groovyx.gvars.scheduler/  Pool, F3Pool, ...           (Java API)
groovyx.gvars.util/      AsyncMessagingCore, ...       (Java API)
groovyx.gvars.util.groovy/ EnhancedRMLock(Closure)    (Groovy sugar)

groovy-parallel-collections/
groovyx.gvars/           ParallelCollections(Function) (Java API)
groovyx.gvars.groovy/    GParsPool, withPool(), categories (Groovy sugar)

groovy-dataflow/
groovyx.gvars.dataflow/  DataflowVariable : CompletionStage (Java API)
groovyx.gvars.dataflow.groovy/ << operator, AwaitableAdapter (Groovy sugar)

groovy-actors/
groovyx.gvars.actor/     Actor, sendAndPromise() : CompletableFuture (Java API)
groovyx.gvars.actor.groovy/ Closure-based react, AwaitableAdapter (Groovy sugar)
groovyx.gvars.activeobject/ @ActiveObject AST transform (Groovy sugar)

groovy-agents/
groovyx.gvars.agent/     Agent(UnaryOperator), getAsync() : CF (Java API)
groovyx.gvars.agent.groovy/ Closure mutations, AwaitableAdapter (Groovy sugar)
```

What Java users see

```
// Parallel collections (no Groovy needed)
var pool = new F3Pool(4);
ParallelCollections.each(pool, items, item -> process(item));
List<R> results = ParallelCollections.collect(pool, items, Item::transform);

// Dataflow (no Groovy needed)
var x = new DataflowVariable<Integer>();
pool.execute(() -> x.bind(42));
x.thenApply(v -> v * 2).thenAccept(System.out::println);

// Actors (no Groovy needed)
var actor = new DynamicDispatchActor(pool) {
    void onMessage(String msg) { reply(msg, toUpperCase()); }
};
CompletableFuture<Object> result = actor.sendAndPromise("hello");
```

What Groovy users see (identical to Option 2)

```
GParsPool.withPool(4) {
    items.eachParallel { process(it) }
}

def x = new DataflowVariable()
async { x << expensiveComputation() }
def result = await x

@ActiveObject
class MyService {
    @ActiveMethod
    def process(data) { /* actor-backed */ }
}
```

What Groovy users see (identical to Option 2)

Pros

- **Broadest audience** — usable from Java, Kotlin, Scala, or any JVM language without the Groovy runtime
- **Groovy users lose nothing** — sugar activates automatically when Groovy is on the classpath; all familiar idioms preserved
- **Same modularity as Option 2** — 5 modules, same decomposition
- **Cleanest async story** — Java layer uses `CompletionStage` (JDK standard, universally composable); Groovy layer bridges to `Awaitable`. No custom `Promise` type needed — fewer adapters, standard types, universal interop
- **Parallel collections can delegate to JDK streams internally** — same benefit as Option 2, and the thin Java API makes the JDK-delegation path natural
- **Future-proof** — pure Java core evolves with the JDK naturally (virtual threads, structured concurrency, future stream enhancements)
- **No version alignment risk** — Java and Groovy APIs are in the same JAR
- **No external dependencies**

Cons

- **Highest refactoring effort** — every `Closure` parameter in the GVars codebase needs a `java.util.function` equivalent; Groovy sugar must be separated into its own package within each module
- **Dual API surface** — Java-functional signatures alongside Groovy-Closure overloads; more surface area to document and test
- **Some GVars patterns don't map cleanly to Java lambdas** — multi-argument closures, `curry()`, delegation strategy; may require workarounds or simplification for the Java API
- **Discipline required** — Java API classes must never reference Groovy types in their signatures, static initializers, or superclasses; enforced by convention (package split) and CI (classpath-exclusion test) rather than by the compiler
- **Java-side parallel collections value is narrow** — Java users may question why they'd use `ParallelCollections.each(pool, items, fn)` over `items.stream().parallel().forEach(fn)`; the answer (pool isolation) is valid but niche. The other modules (dataflow, actors, agents) don't have this problem — no JDK equivalent exists

Async integration approach

Java API types return `CompletableFuture<T>` or implement `CompletionStage<T>`. The Groovy async proposal almost certainly ships with a built-in `AwaitableAdapter` for `CompletionStage`, meaning GVars-derived modules get `await` support for free with zero custom adapter code. If additional convenience is wanted (e.g., `<<` operator on `DataflowVariable`, for `await` on channels), those live in the Groovy sugar layer.

Side-by-Side Comparison

Criterion	Option 1: Monolith	Option 2: Groovy Modules	Option 3: Java-First
Modularity	None — all or nothing	5 focused modules	5 focused modules
Java usability	Needs Groovy; Closure API	Needs Groovy; Closure API	Pure Java; java.util.function API
Groovy experience	Full GVars idioms	Full GVars idioms	Full GVars idioms (automatic)
Kotlin/Scala/other JVM	Awkward (Closure interop)	Awkward (Closure interop)	Natural (java.util.function)
Async/Awaitable	Bolted-on adapters	Deep native integration	Layered: CompletionStage + auto adapter
Parallel collections impl	Port GVars code	Port or delegate to JDK streams	Delegate to JDK streams (natural fit)
Migration effort	Low	Medium	High
Ongoing maintenance	Single API surface	Single API surface per module	Dual API surface per module
External dependencies	None (after pruning)	None	None
JDK future-proofing	Indirect (Groovy mediates)	Indirect (Groovy mediates)	Direct (pure Java core)
Number of modules	1	5	5
@Parallel enhancement	Separable — works the same under all options		

When To Choose Each

If your priority is...	Choose
Ship fast, minimal effort, validate demand	Option 1 — bring it in, prune dead weight, ship. Can always decompose later if there's uptake.
Best Groovy experience, deepest async integration, Groovy-first community	Option 2 — native <code>Awaitable</code> , clean Groovy idioms, medium effort. The pragmatic choice if the primary audience is Groovy developers.
Broadest JVM audience, cleanest architecture, long-term investment	Option 3 — Java-first with automatic Groovy sugar. The most work upfront but the most durable result. Particularly compelling for dataflow and actors, which have no JDK equivalent and would attract Java/Kotlin users on their own merits.

Pragmatic middle ground: Start with Option 2, but design the internal implementations with `java.util.function` interfaces from day one (have the Closure-based API delegate to function-based internals). This makes a future evolution to Option 3 straightforward — you'd be extracting already-clean Java internals and adding the `compileOnly` Groovy boundary, not rewriting from scratch.